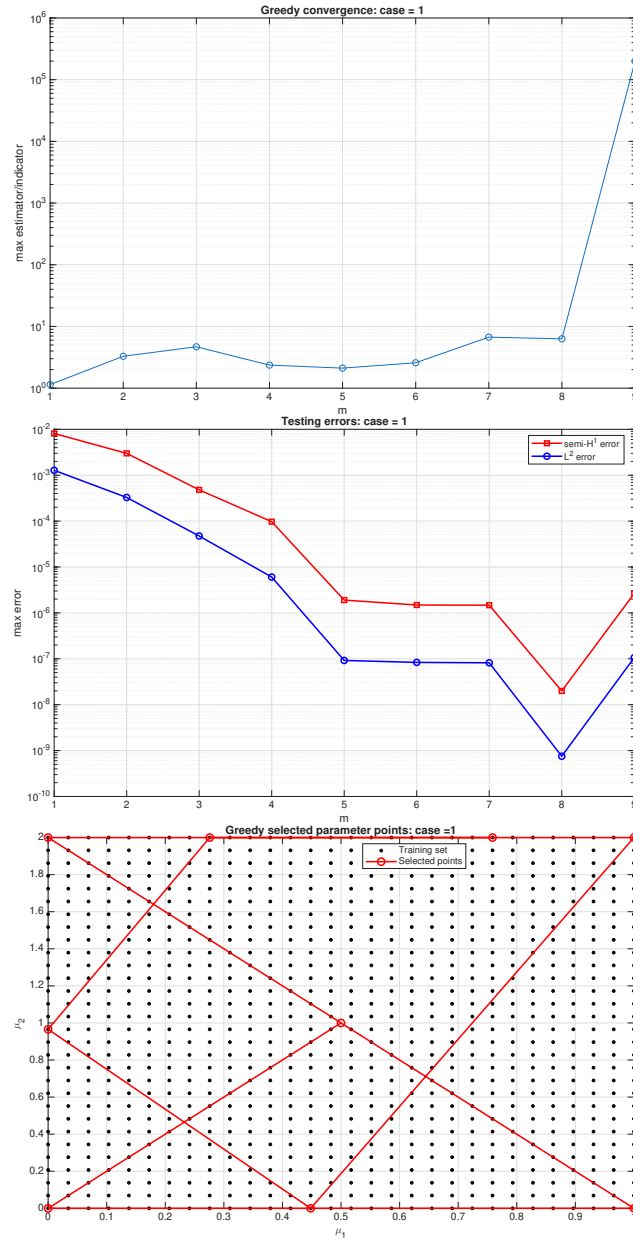
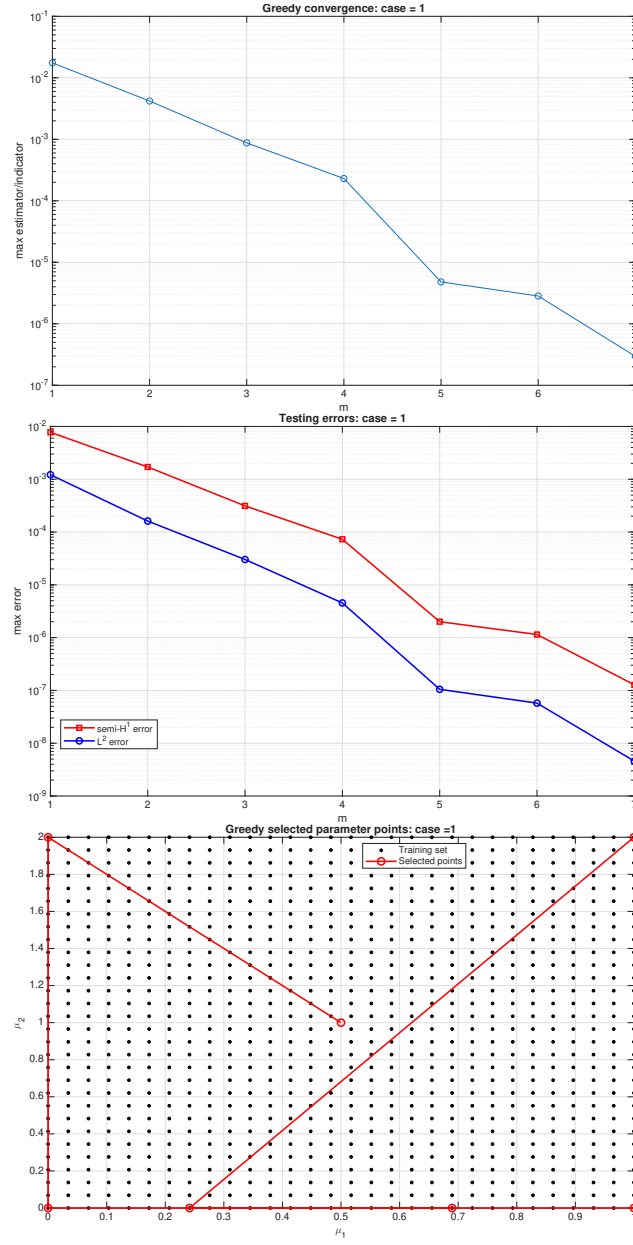


HW4(FEA): Appiah Francis

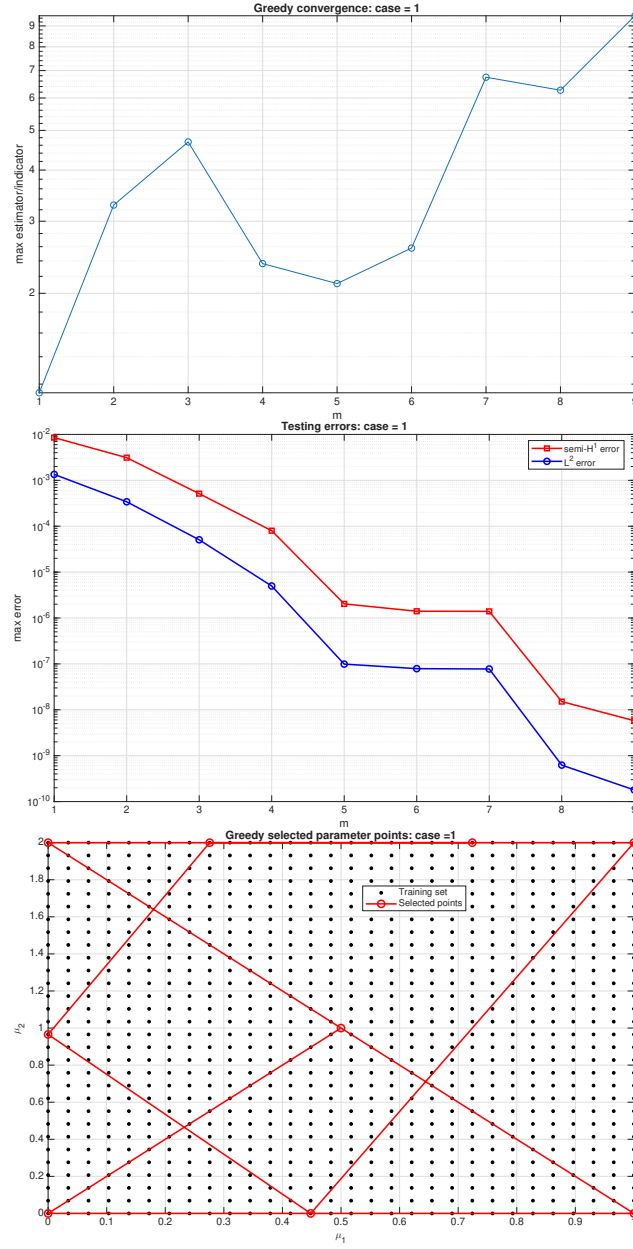
Case 1: RB space $V_m = \text{span}\{U_h(\mu_j), j = 1, \dots, m\}$, with L_1 error indicator



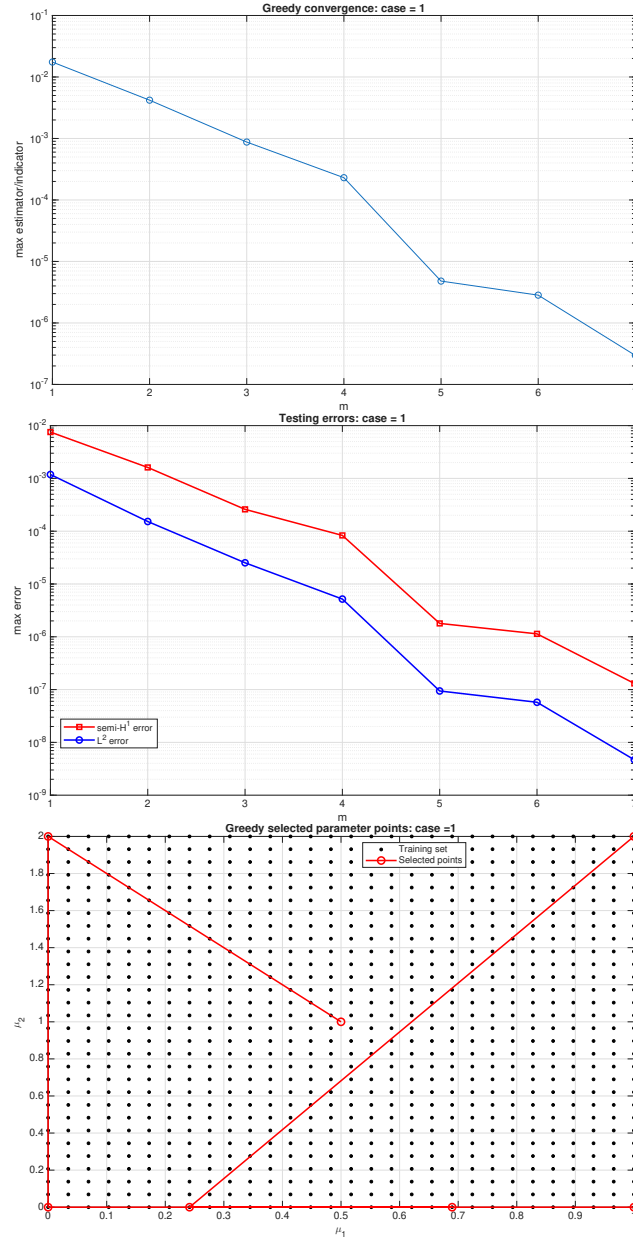
Case 1: RB space $V_m = \text{span}\{U_h(\mu_j), j = 1, \dots, m\}$, with residual error estimator



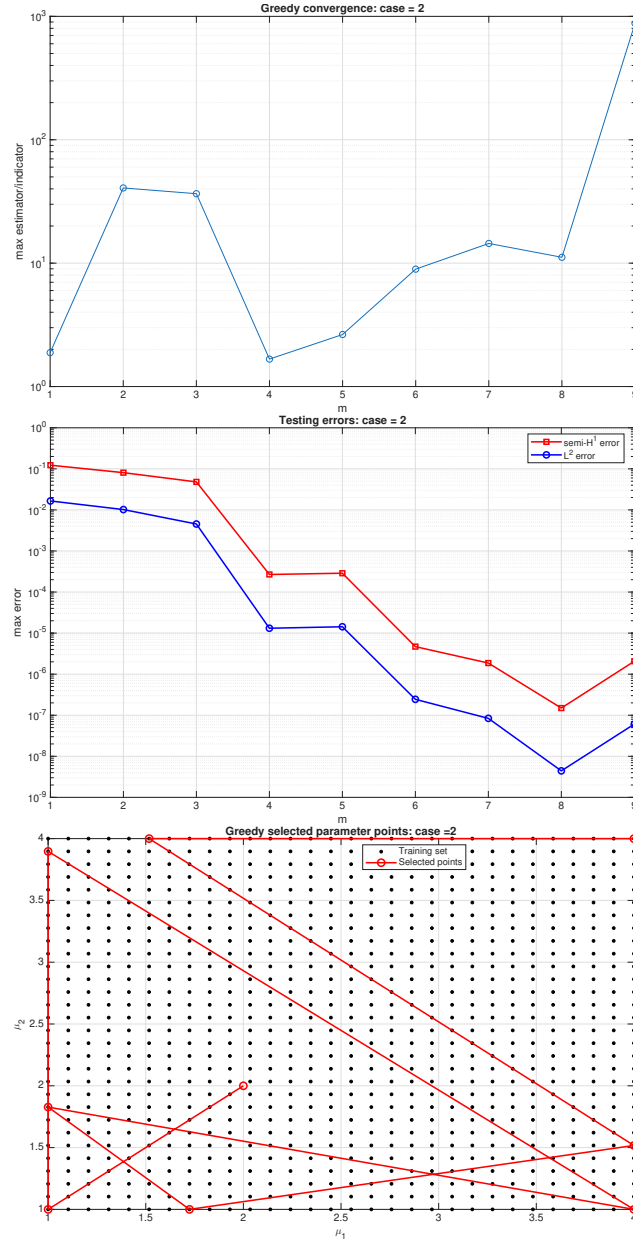
Case 1: QR factorization, with L_1 error indicator



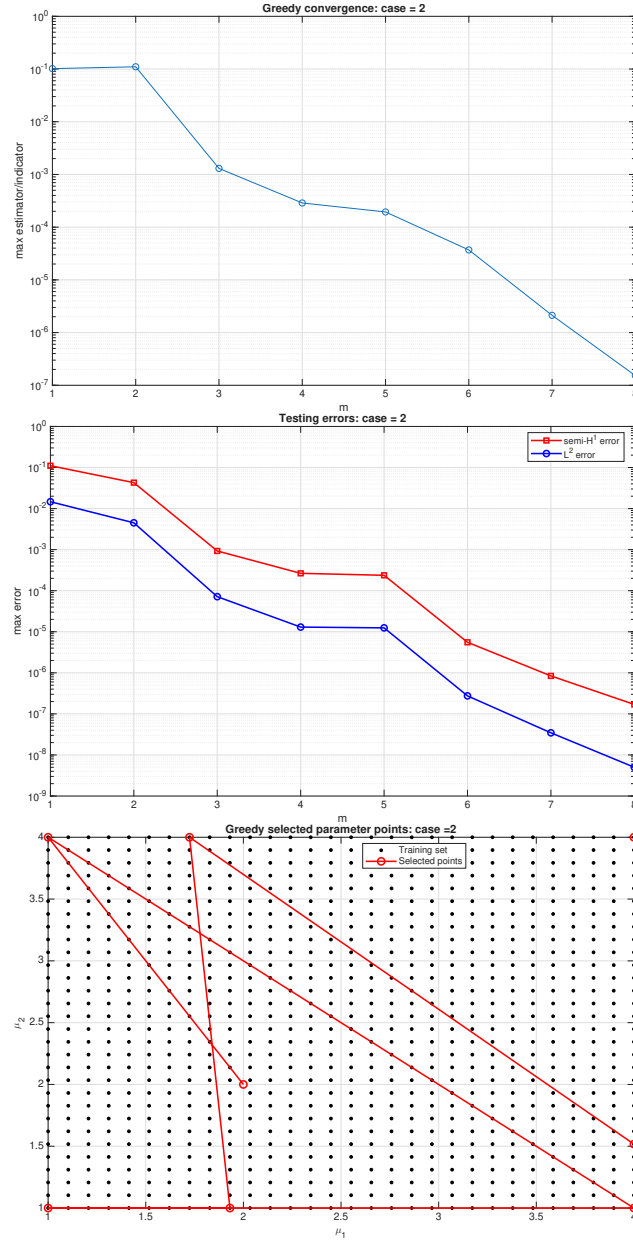
Case 1: QR factorization, with residual error estimator



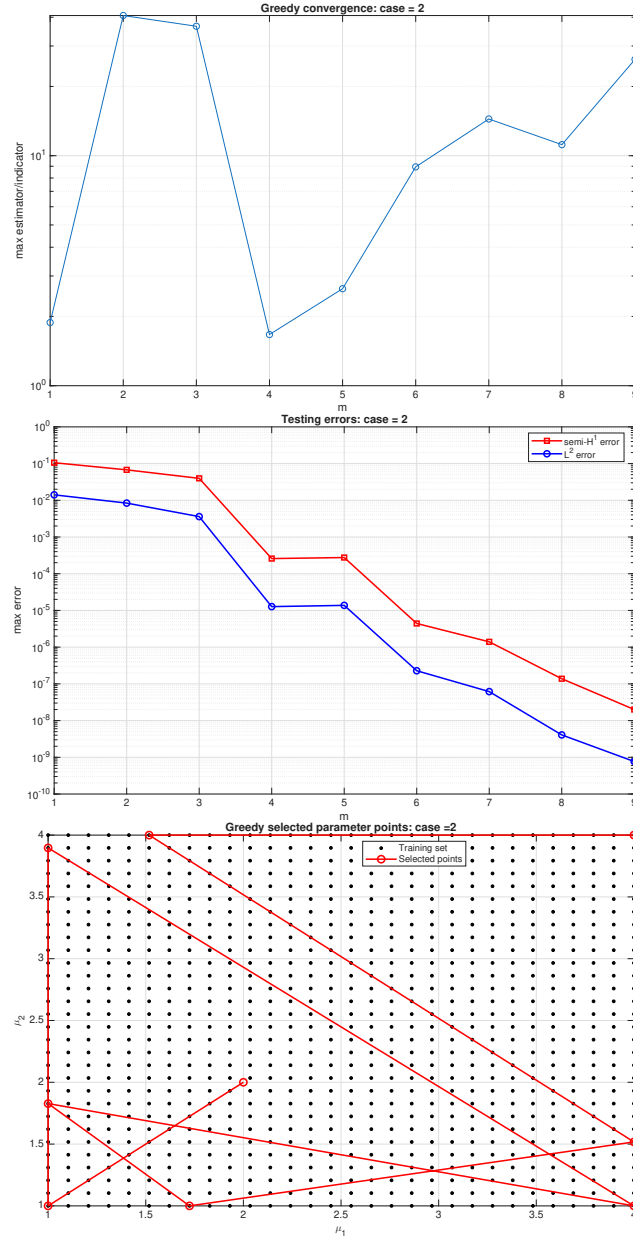
Case 2: RB space $V_m = \text{span}\{U_h(\mu_j), j = 1, \dots, m\}$, with L_1 error indicator



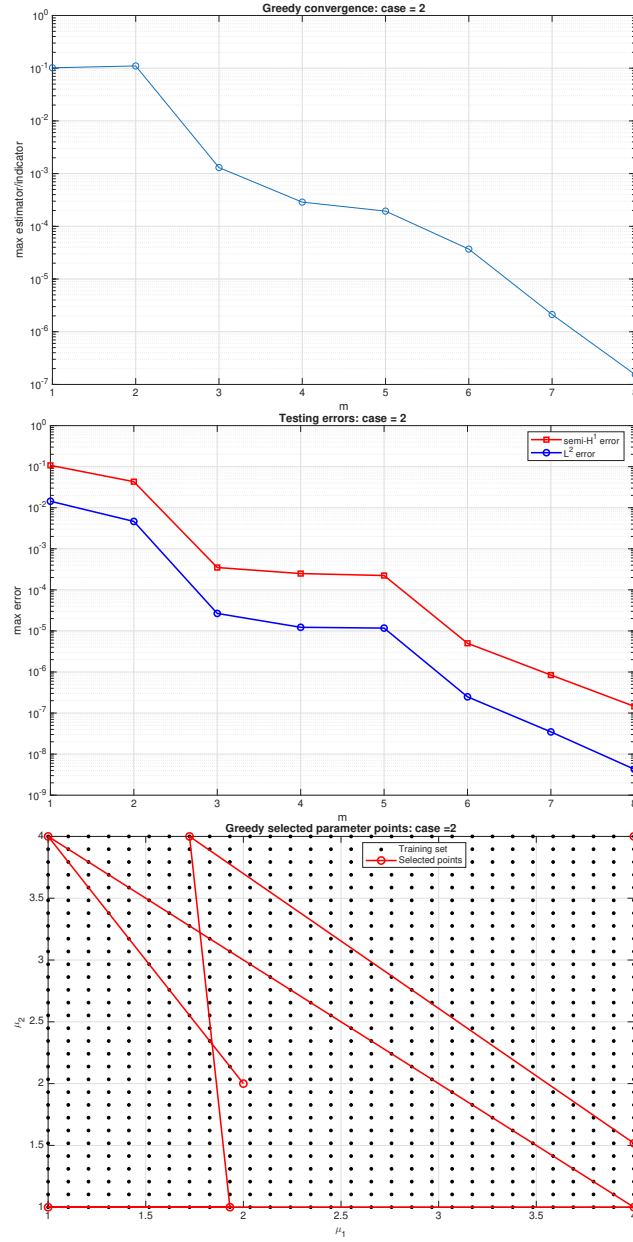
Case 2: RB space $V_m = \text{span}\{U_h(\mu_j), j = 1, \dots, m\}$, with residual error estimator



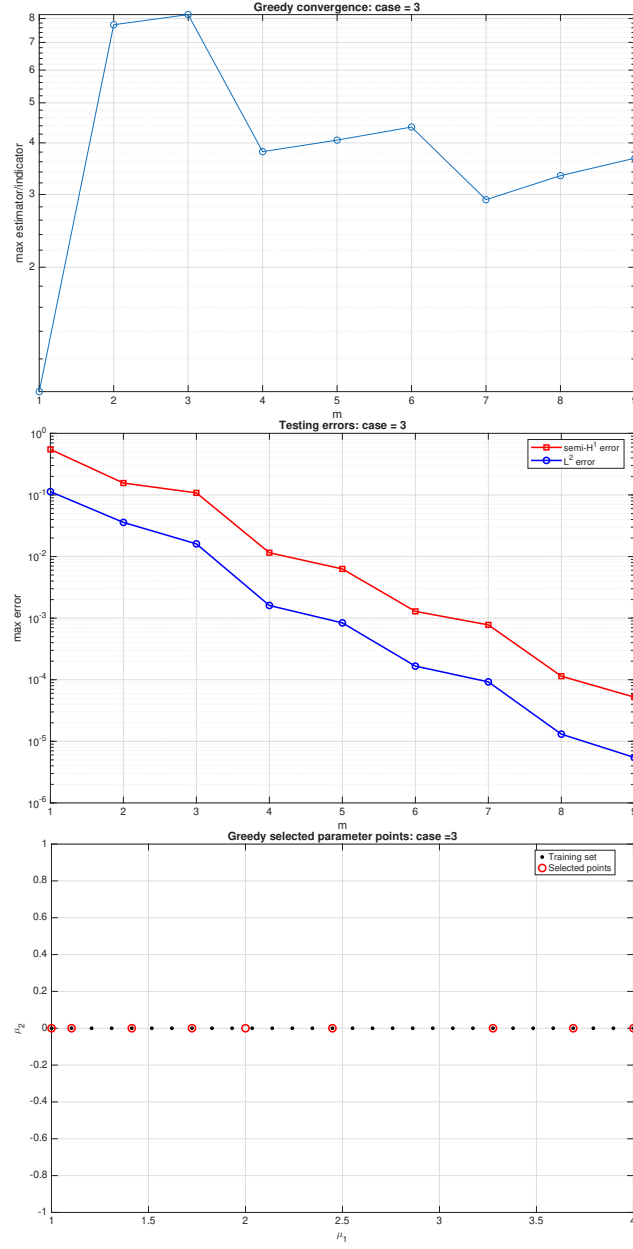
Case 2: QR factorization, with L_1 error indicator



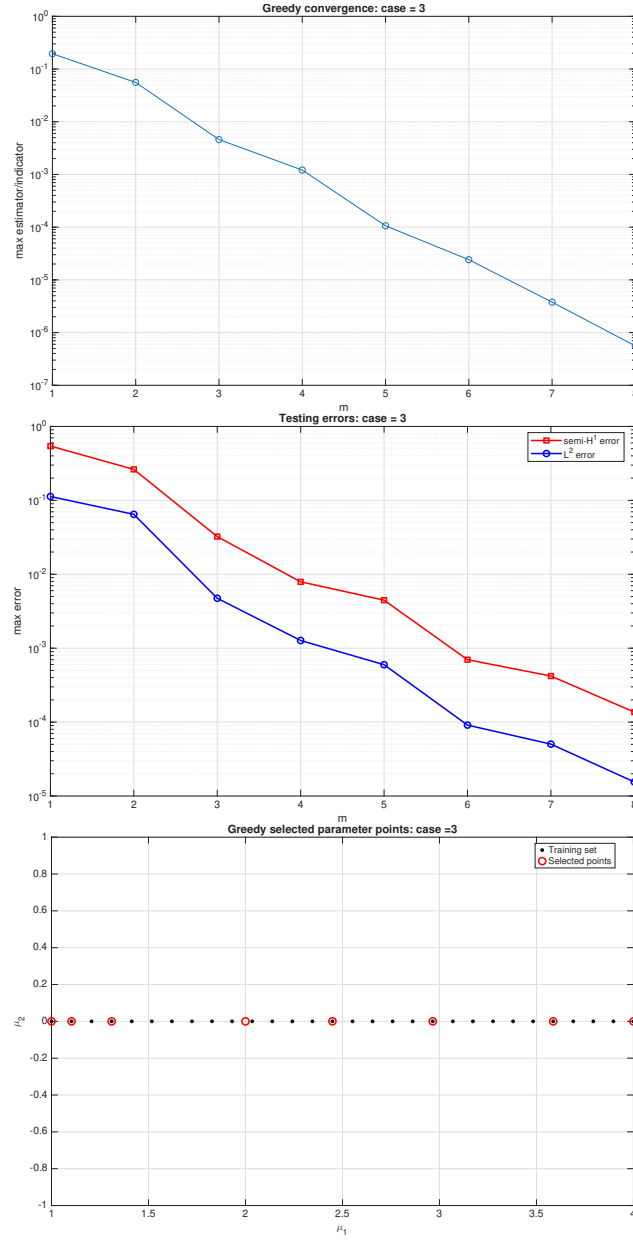
Case 2: QR factorization, with residual error estimator



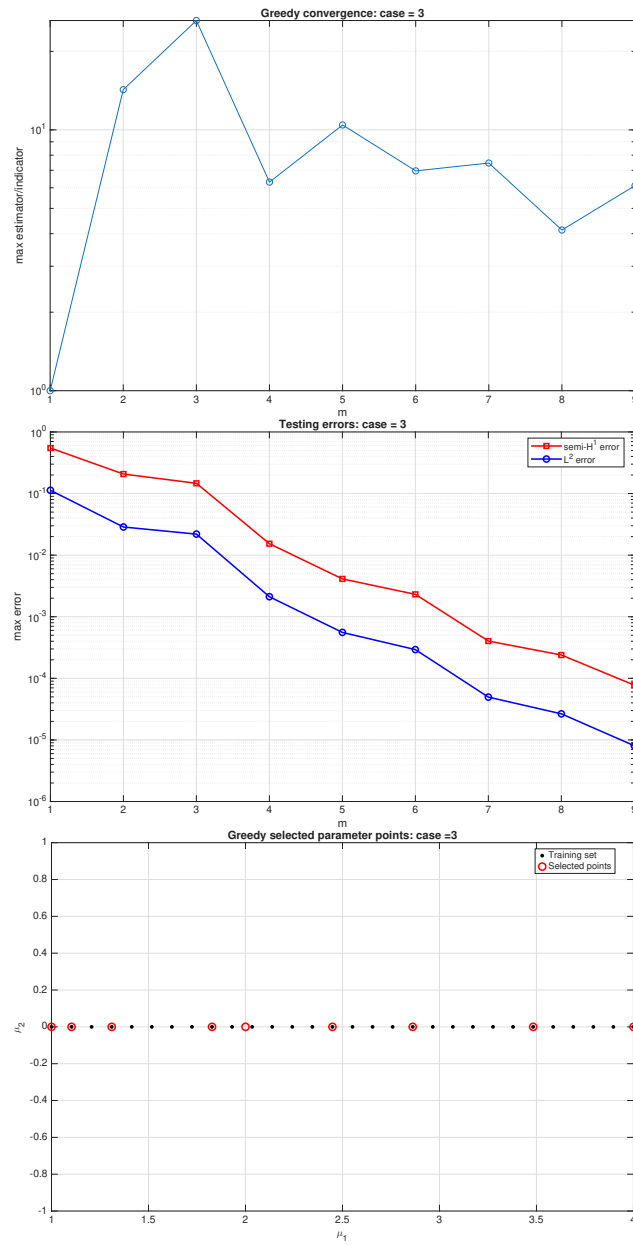
Case 3: RB space $V_m = \text{span}\{U_h(\mu_j), j = 1, \dots, m\}$, with L_1 error indicator



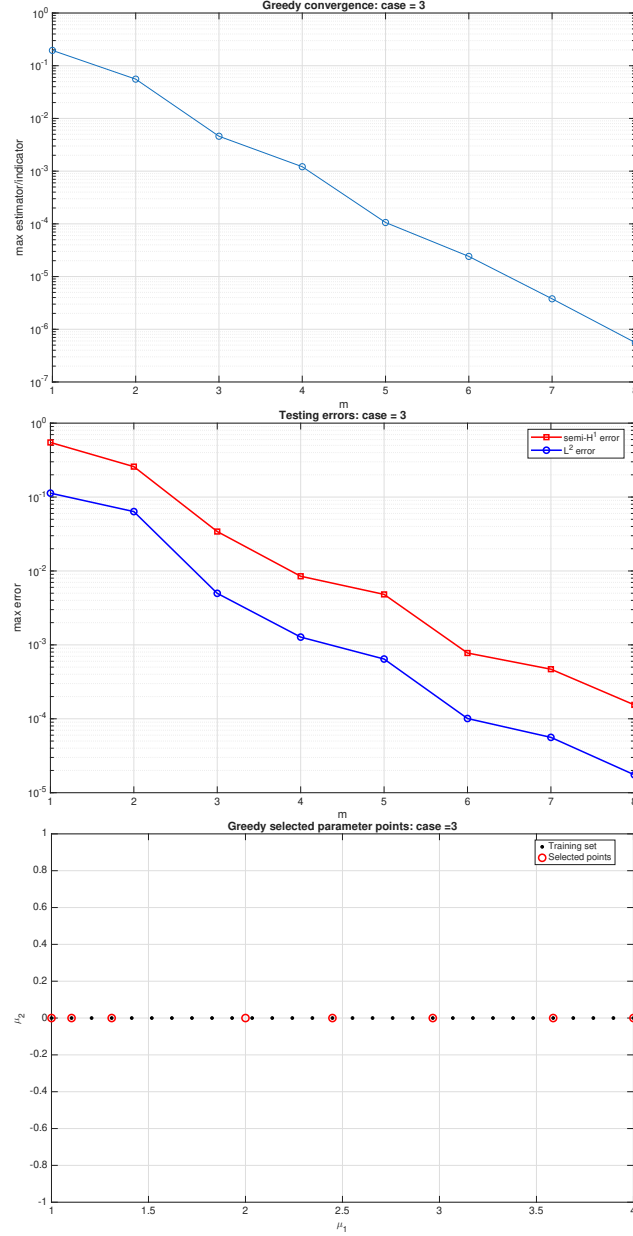
Case 3: RB space $V_m = \text{span}\{U_h(\mu_j), j = 1, \dots, m\}$, with residual error estimator



Case 3: QR factorization, with L_1 error indicator



Case 3: QR factorization, with residual error estimator



conclusion

The Reduced Basis Method (RBM) was tested using the finite element method (FEM) solution as the high-fidelity reference across three different parameterized cases. For each case, four configurations were considered:

- Standard RB space with ℓ_1 -indicator,
- Standard RB space with residual-based estimator,
- QR-orthonormalized basis with ℓ_1 -indicator,

- QR-orthonormalized basis with residual-based estimator.

Across all cases, the greedy algorithm exhibits a monotonic decay of testing errors. The greedy algorithm selects parameter points that are well distributed over the parameter domain. The selected points tend to concentrate in regions where the solution exhibits stronger variation, demonstrating the adaptive nature of the method. The residual-based estimator shows faster and more stable convergence, typically reaching the prescribed tolerance with fewer basis functions (approximately $m \approx 8$). In contrast, the ℓ_1 -based indicator converges more slowly and may require a larger reduced basis dimension to achieve comparable accuracy. I set the maximum dimension of the reduced basis to 9. That's why the $m^* = 9$ for ℓ_1 error indicator. The plot for the greedy convergence for ℓ_1 error indicator does not tell us much about how the testing errors, rather it helped to select the next parameter to use in building the RB basis.

The semi- H_1 and L_2 errors decrease rapidly as the reduced basis dimension increases. The decay is approximately exponential. The residual-based greedy approach consistently yields lower errors for a given basis size. The use of QR factorization improves numerical stability and conditioning of the reduced system, although it does not significantly affect the convergence rate.

Overall, the RBM significantly reduces computational cost while maintaining high accuracy, making it well-suited for repeated evaluations in parametrized problems.

Listing 1: main.m

```

1 close all; clc;
2 caseType = 3;
3 basisType = 2;      % 1 Um ; 2 QR
4 ErrEstimator = 1;   % 1 l1-norm ; 2 residual
5 a = 0; b = 1;
6 xm = 0.5;
7 sigmaLB = 1.0;
8 N = 159;            % choose even so x=0.5 is a node
9 maxRB = 9;          % dim of Reduce Basis
10 nTest = 100;
11 tolGreedy = 1e-6;
12 x = linspace(a,b,N+2)';
13 h = (b-a)/(N+1);
14 %% Training set
15 n1 = 30; n2 = 30;
16 if caseType == 1
17     sqL = 0; sqR = 1; sqB = 0; sqT = 2;
18 elseif caseType == 2
19     sqL = 1; sqR = 4; sqB = 1; sqT = 4;
20 elseif caseType == 3
21     sqL = 1; sqR = 4;
22 end
23 mu1_vals = linspace(sqL,sqR,n1);
24 if caseType == 3
25     Dtrain = mu1_vals(:);
26     Dtest = sqL + sqR*rand(nTest,1);
27 else
28     mu2_vals = linspace(sqB,sqT,n2);
29     [mu1, mu2] = meshgrid(mu1_vals,mu2_vals);
30     Dtrain = [mu1(:), mu2(:)];
31     Dtest = [sqL + sqR*rand(nTest,1), sqB + sqT*rand(nTest,1)];
32 end

```

```

33 %% Assemble parameter-independent matrices
34 if caseType == 1
35     [A0, A1, A2, M, K, F] = MatrixCase1(N, x);
36 elseif caseType == 2
37     [A0, A1, A2, M, K, F] = MatrixCase2(N,x);
38 elseif caseType == 3
39     [A0, A1, A2, M, K, F] = MatrixCase3(N,x);
40 end
41 %% INITIAL BASIS
42 if caseType == 3
43     Initmu = 1/2*sqR;
44 else
45     Initmu = [2,2];           % row vector
46 end
47 Uh = solveFEM(Initmu, A0, A1, A2, F,caseType);
48 Um = Uh;                     % normalize
49 selected_mu = Initmu;        % store selected parameters
50 Delta_history = [];
51
52 %% GREEDY ALGORITHM
53 errH1 = zeros(maxRB+1,1);
54 errL2 = zeros(maxRB+1,1);
55 for m = 1:maxRB
56     if basisType == 2
57         [Qm, Rm] = qr(Um,0);
58     else
59         Qm = Um;
60     end
61     Delta = zeros(size(Dtrain,1),1);
62
63     for i = 1:size(Dtrain,1)
64         mu = Dtrain(i,:);
65
66         % Skip already selected parameters
67         if ismember(mu, selected_mu, 'rows')
68             Delta(i) = -inf;
69             continue;
70         end
71
72         % ROM solve
73         [u_rom,c] = solveROM(mu, Qm, A0, A1, A2, F,caseType);
74
75         if ErrEstimator == 1
76             % L1 indicator
77             if basisType == 1
78                 Delta(i) = norm(c,1);
79             else
80                 Delta(i) = norm(Rm\c,1);
81             end
82         else
83             % --- alternatively (better) ---
84             r = computeResidual(mu, u_rom, A0, A1, A2, K, F,caseType);
85             Delta(i) = r;
86         end
87     end
88
89     % Find worst parameter
90     [maxDelta, idx] = max(Delta);
91     Delta_history = [Delta_history; maxDelta];

```

```

92
93 fprintf('m=%d,max_indicator=%.3e\n', m, maxDelta);
94
95 if (maxDelta < tolGreedy) || (m == maxRB)
96     % Compute error for current basis BEFORE breaking
97
98     fprintf('\nComputing_testing_errors(final_step)...\n');
99
100    V = Qm; % use current basis (already size m)
101
102    maxErrH1 = 0;
103    maxErrL2 = 0;
104
105    for k = 1:nTest
106        mutest = Dtest(k,:);
107        UFOM = solveFEM(mutest, A0, A1, A2, F,caseType);
108        UROM = solveROM(mutest, V, A0, A1, A2, F, caseType);
109
110        e = UFOM - UROM;
111
112        eH1 = sqrt(e' * K * e);
113        eL2 = sqrt(e' * M * e);
114
115        maxErrH1 = max(maxErrH1, eH1);
116        maxErrL2 = max(maxErrL2, eL2);
117    end
118
119    errH1(m) = maxErrH1;
120    errL2(m) = maxErrL2;
121
122    fprintf('m=%2d(final),H1=%.4e,L2=%.4e\n', ...
123        m, errH1(m), errL2(m));
124
125    break;
126 end
127
128 % New parameter
129 mu_new = Dtrain(idx,:);
130
131 % Store it
132 selected_mu = [selected_mu; mu_new];
133
134 % Full solve
135 u_new = solveFEM(mu_new, A0, A1, A2, F,caseType);
136
137 % Enrich basis
138 Um = [Um, u_new];
139
140 %% -----
141 % Testing errors
142 %% -----
143 fprintf('\nComputing_testing_errors...\n');
144
145 V = Qm;
146 %[Qm, ~] = qr(Um,0);
147
148 maxErrH1 = 0;
149 maxErrL2 = 0;
150

```

```

151     for k = 1:nTest
152         mutest = Dtest(k,:);
153         UFOM = solveFEM(mutest, A0, A1, A2, F,caseType);
154
155         UROM = solveROM(mutest, V, A0, A1, A2, F, caseType);
156
157         e = UFOM - UROM;
158
159         eH1 = sqrt(e' * K * e);
160         eL2 = sqrt(e' * M * e);
161
162         maxErrH1 = max(maxErrH1, eH1);
163         maxErrL2 = max(maxErrL2, eL2);
164     end
165
166     errH1(m) = maxErrH1;
167     errL2(m) = maxErrL2;
168
169     fprintf('m=%2d, max H1-semi error=%.4e, max L2 error=%.4e\n', ...
170         m, errH1(m), errL2(m));
171 end
172 mStar = size(Um,2);
173 fprintf('\nOffline stage finished with m*=%d.\n', mStar);
174 %% Plot convergence
175 figure;
176 semilogy(1:length(Delta_history), Delta_history, '-o');
177 xlabel('m'); ylabel('max estimator/indicator');
178 title(sprintf('Greedy convergence: case=%d', caseType))
179 grid on;
180 filename = sprintf('img/GC_case%d_basis%d_err%d.eps', caseType, basisType, ErrEstimator);
181 print('-depsc2', filename);
182
183 figure;
184 semilogy(1:mStar, errH1(1:mStar), 'rs-', 'LineWidth', 1.5); hold on;
185 semilogy(1:mStar, errL2(1:mStar), 'bo-', 'LineWidth', 1.5);
186 grid on;
187 xlabel('m');
188 ylabel('max error');
189 title(sprintf('Testing errors: case=%d', caseType))
190 legend('semi-H^1 error', 'L^2 error', 'Location', 'best');
191 filename = sprintf('img/TE_case%d_basis%d_err%d.eps', caseType, basisType, ErrEstimator);
192 print('-depsc2', filename);
193
194 figure;
195 if caseType == 3
196     plot(Dtrain, 0*Dtrain, 'k.', 'MarkerSize', 8); hold on;
197     plot(selected_mu, 0*selected_mu, 'ro', 'LineWidth', 1.5, 'MarkerSize', 7);
198 else
199     plot(Dtrain(:,1), Dtrain(:,2), 'k.', 'MarkerSize', 8); hold on;
200     plot(selected_mu(:,1), selected_mu(:,2), 'ro', 'LineWidth', 1.5, 'MarkerSize', 7);
201 end
202 grid on;
203 xlabel('\mu_1');
204 ylabel('\mu_2');
205 title(sprintf('Greedy selected parameter points: case=%d', caseType));
206 legend('Training set', 'Selected points', 'Location', 'best');
207 filename = sprintf('img/GSP_case%d_basis%d_err%d.eps', caseType, basisType, ErrEstimator);
208 print('-depsc2', filename);
209 %% -----

```



```

210 fprintf('\n=====SUMMARY=====\\n');
211 fprintf('%d\\n', caseType);
212 fprintf('Final_RB_dimension_m*UUUUUUUUUU=\\n', mStar);
213 fprintf('Final_greedy_indicatorUUUUUUUUUU=\\n', Delta_history(end));
214 fprintf('Final_max_test_semi-H1_errorUUUU=\\n', errH1(mStar));
215 fprintf('Final_max_test_L2_errorUUUUUUUU=\\n', errL2(mStar));
216 fprintf('=====\\n');

```

Listing 2: solveFEM.m

```

1 function u = solveFEM(mu, A0, A1, A2, F, caseType)
2 if caseType == 1
3     mu1 = mu(1); mu2 = mu(2);
4     A = 2*A0 + mu1*A1 + mu2*A2;
5 elseif caseType == 2
6     mu1 = mu(1); mu2 = mu(2);
7     A = mu1*A0 + mu2* A1 + A2;
8 else
9     A = 0.1*A0 + mu*A1 + A2;
10 end
11 u = A \ F;
12 end

```

Listing 3: solveROM.m

```

1 function [u_rom,c] = solveROM(mu, V, A0, A1, A2, F, caseType)
2 if caseType == 1
3     mu1 = mu(1); mu2 = mu(2);
4     A = 2*A0 + mu1*A1 + mu2*A2;
5 elseif caseType == 2
6     mu1 = mu(1); mu2 = mu(2);
7     A = mu1*A0 + mu2* A1 + A2;
8 else
9     A = 0.1*A0 + mu*A1 + A2;
10 end
11
12 Ar = V'*A*V;
13 fr = V'*F;
14
15 c = Ar \ fr;
16 u_rom = V*c;
17 end

```

Listing 4: MatrixCase1.m

```

1 function [A0, A1, A2, M, K, F] = assemble_matrices(N, x)
2
3 A0 = sparse(N,N);
4 A1 = sparse(N,N);
5 A2 = sparse(N,N);
6 M = sparse(N,N);
7 F = zeros(N,1);
8
9 for k = 1:N+1
10
11     h = x(k+1) - x(k);
12
13     % Local stiffness

```

```

14 Ke = (1/h)*[1 -1; -1 1];
15
16 % Local sin-weighted stiffness
17 %I = integral(@(xx) sin(xx), x(k), x(k+1));
18 I = cos(x(k)) - cos(x(k+1));
19 Ke1 = (1/h^2)*I * [1 -1; -1 1];
20
21 Mele = h/6*[2 1; 1 2];
22
23 % Local mass (reaction)
24 % --- Reaction matrix using 2-point Gaussian quadrature ---
25 Me = zeros(2,2);
26 % Quadrature points (reference)
27 xi = [-1/sqrt(3), 1/sqrt(3)];
28
29 for q = 1:2
30
31     % Map to physical element
32     xq = (x(k) + x(k+1))/2 + (h/2)*xi(q);
33
34     % Only integrate on [0, 0.5]
35     if xq <= 0.5 + 1e-12
36
37         w = exp(xq);
38
39         % Shape functions
40         phi = [(1 - xi(q))/2; (1 + xi(q))/2];
41
42         % Add contribution
43         Me = Me + w * (phi * phi') * (h/2);
44     end
45 end
46
47 % Load vector (f=1)
48 Fe = (h/2)*[1;1];
49
50 % Assembly
51 nodes = [k-1, k];
52 for ai = 1:2 %interior indexing
53     i = nodes(ai);
54     for bj = 1:2
55         j = nodes(bj);
56         if i >=1 && j >=1 && i <=N && j <=N
57             A0(i,j) = A0(i,j) + Ke(ai,bj);
58             A1(i,j) = A1(i,j) + Ke1(ai,bj);
59             A2(i,j) = A2(i,j) + Me(ai,bj);
60             M(i,j) = M(i,j) + Mele(ai,bj);
61         end
62     end
63     if i >=1 && i <=N
64         F(i) = F(i) + Fe(ai);
65     end
66 end
67 end
68 K = A0;
69 end

```

Listing 5: MatrixCase2.m

```

1 function [A0_l, A1_r, A2, M,K, F] = MatrixCase2(N, x)
2
3 A0_l = sparse(N,N);
4 A1_r = sparse(N,N);
5 A2 = sparse(N,N);
6 M = sparse(N,N);
7 K = sparse(N,N);
8 F = zeros(N,1);
9
10 for k = 1:N+1
11     h = x(k+1) - x(k);
12     xl = x(k); xr = x(k+1);
13     % Local stiffness
14     Ke = (1/h)*[1 -1; -1 1];
15     % Local mass
16     Mele = h/6*[2 1; 1 2];
17
18     A0_le = zeros(2,2);
19     A1_re = zeros(2,2);
20     if xr <= 0.5
21         % Local sin-weighted stiffness
22         I = cos(x(k)) - cos(x(k+1));
23         Ke1 = (1/h^2)*I * [1 -1; -1 1];
24         A0_le = 2*Ke + Ke1;
25     elseif xl >= 0.5
26         I = 1/3*(x(k+1)^3 - x(k)^3);
27         Ke1 = (1/h^2)*I * [1 -1; -1 1];
28         A1_re = Ke1;
29     else
30         error('Choose mesh so x=0.5 is a node. ');
31     end
32
33     % --- Reaction matrix using 2-point Gaussian quadrature ---
34     Me = zeros(2,2);
35     % Quadrature points (reference)
36     xi = [-1/sqrt(3), 1/sqrt(3)];
37     for q = 1:2
38         % Map to physical element
39         xq = (x(k) + x(k+1))/2 + (h/2)*xi(q);
40         w = exp(xq);
41         % Shape functions
42         phi = [(1 - xi(q))/2; (1 + xi(q))/2];
43         % Add contribution
44         Me = Me + w * (phi * phi') * (h/2);
45     end
46
47     % Load vector (f=1)
48     Fe = (h/2)*[1;1];
49
50     % Assembly
51     nodes = [k-1, k];
52     for ai = 1:2 %interior indexing
53         i = nodes(ai);
54         for bj = 1:2
55             j = nodes(bj);
56             if i >= 1 && j >= 1 && i <= N && j <= N
57                 A0_l(i,j) = A0_l(i,j) + A0_le(ai,bj);
58                 A1_r(i,j) = A1_r(i,j) + A1_re(ai,bj);

```

```

59         A2(i,j) = A2(i,j) + Me(ai,bj);
60         M(i,j) = M(i,j) + Mele(ai,bj);
61         K(i,j) = K(i,j) + Ke(ai,bj);
62     end
63 end
64 if i >=1 && i <=N
65     F(i) = F(i) + Fe(ai);
66 end
67 end
68 end
69
70 end

```

Listing 6: MatrixCase3.m

```

1 function [A0, A1, A2, M,K, F] = MatrixCase3(N, x)
2
3 A0 = sparse(N,N);
4 A1 = sparse(N,N);
5 A2 = sparse(N,N);
6 M = sparse(N,N);
7 K = sparse(N,N);
8 F = zeros(N,1);
9
10 for k = 1:N+1
11     h = x(k+1) - x(k);
12     % Local stiffness
13     Ke = (1/h)*[1 -1; -1 1];
14     % Local mass
15     Me = h/6*[2 1;1 2];
16     % Local advection
17     Ade = 1/2*[-1 1;-1 1];
18
19     % Load vector (f=1)
20     Fe = (h/2)*[1;1];
21
22     % Assembly
23     nodes = [k-1, k];
24     for ai = 1:2 %interior indexing
25         i = nodes(ai);
26         for bj = 1:2
27             j = nodes(bj);
28             if i >=1 && j >=1 && i <=N && j <=N
29                 A0(i,j) = A0(i,j) + Ke(ai,bj);
30                 A1(i,j) = A1(i,j) + Ade(ai,bj);
31                 A2(i,j) = A2(i,j) + Me(ai,bj);
32                 M(i,j) = M(i,j) + Me(ai,bj);
33                 K(i,j) = K(i,j) + Ke(ai,bj);
34             end
35         end
36         if i >=1 && i <=N
37             F(i) = F(i) + Fe(ai);
38         end
39     end
40 end
41
42 end

```